



江家德

AI模型架構設計

物聯網自動化

強化學習

網路路由智慧化

國立成功大學 | 資訊工程學系 碩士畢業

彰化縣線西鄉 | 1年(含)以下工作經歷 | 希望職稱：員工

邏輯清楚，善於簡化複雜概念、溝通與合作。

對待工作當責、且具上進心，不斷學習新知與技能。

個人資料 男、25歲、屆退(2026/2)
就業狀態 待業中
主要手機 0906-072-132
E-mail angel606901@gmail.com
通訊地址 彰化縣線西鄉線西村***
聯絡方式 手機連絡，09:00~21:00
駕駛執照 普通重型機車駕照
交通工具 普通重型機車

學歷

國立成功大學

2023/9~2025/7

資訊工程學系 | 碩士畢業

國立東華大學

2019/9~2023/6

物理學系 | 大學畢業

工作經驗

總年資 1年(含)以下工作經歷

求職條件

希望性質 全職工作
上班時段 日班
可上班日 錄取後兩週可上班
希望待遇 面議
希望地點 桃園市、宜蘭縣、新竹縣市、台中市、台南市、高雄市
遠端工作 對遠端工作有意願

希望職稱 員工

希望職類 其他特殊工程師、AI工程師、演算法工程師、軟體工程師

工作內容 網路路由智慧自動化AI設計, 自動化控制AI設計, AIoT, 使用 DRL 實現 LLM 語言模型 deep thinking, AI模型設計, 等等AI方面
軟韌體設計

語言能力

英文

聽/中等 | 說/中等 | 讀/中等 | 寫/中等

專長

基礎語言及框架

強化學習

深度學習

C++

C

Python

Java

#Python #C++ #Java #Machine Learning

進修課程專案-股票投資機器人實作

期末實作專案： 股票投資機器人實作

- 各組需自行建置機器人程式，包含 [Deep Learning 模型](#) 與 [資產配置模組](#)，每日產生交易訂單到本課程股市交易模擬平台進行交易。
- 機器人交易目標：**最大化獲利**。(通用)
 - 每一組預設會給 100,000,000 元(虛擬金額)當作基金

2025/3~2025/7

【專案描述】

開發基於深度強化學習 (Deep Reinforcement Learning) 的自動化投資代理人，旨在解決多檔股票的動態資產配置問題。不同於傳統量化策略，模型透過與模擬環境的互動，自主學習出最大化長期回報的交易策略。

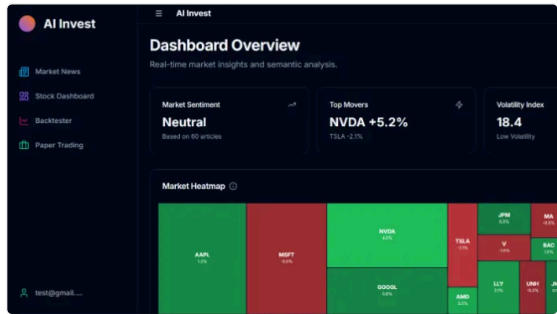
【模型與演算法架構】

核心演算法：實作 Policy Gradient (REINFORCE) 策略梯度演算法，讓 Agent 直接學習最佳策略機率分佈，實現端到端的買賣決策。
神經網路設計：採用 LSTM (Long Short-Term Memory) 作為 Policy Network 的核心，有效捕捉股價與技術指標 (MACD, RSI, ATR, CCI) 的長短期時間序列特徵。
決策空間：模型不僅決定買賣方向，更能處理離散化的動態倉位管理 (決定買賣多少單位)，實現精細的資金控管。

【訓練環境建置】

開發相容於 OpenAI Gym 介面的 StockTradingEnvMulti 模擬環境，支援多股並行交易。
設計包含交易手續費與時間衰減成本 (Time Decay) 的獎勵機制 (Reward Engineering)，引導模型避免過度頻繁交易或無效持倉，專注於真實獲利。

[前往查看](#) >



AI 驅動投資情報分析平台

2025/12~2025/12

【專案概述】

開發一套整合即時股市數據與生成式 AI 的全端投資分析平台。解決了傳統量化工具缺乏語意理解、以及通用 LLM 無法獲取即時市場資訊的痛點。

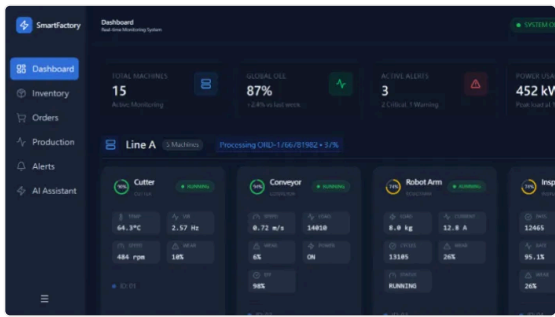
【技術架構】

- Frontend: Next.js 14 (App Router), TypeScript, Tailwind CSS, Shadcn/UI, Recharts, Framer Motion
- Backend: Python FastAPI (Async), SQLAlchemy, Pydantic
- Database & AI: PostgreSQL, Google Gemini API (RAG), Playwright (Crawler)
- DevOps: Docker, Vercel, Render, Git/GitHub。

【關鍵貢獻與技術亮點】

- 建構 Context-Aware RAG (檢索增強生成) 系統：
 - 設計 RAG Pipeline，將即時爬取的財經新聞與情緒指標動態注入 LLM Context，解決了通用 AI 模型無法理解「今日市場」與產生幻覺的問題。
 - 整合 Google Gemini Pro 模型，實現自然語言查詢投資建議與財報解讀。
- 高強韌性分散式爬蟲 (Resilient Data Engineering)：
 - 使用 Playwright 與 AsyncIO 開發非同步爬蟲，結合 User-Agent 輪替與指數退避 (Exponential Backoff) 機制，成功繞過 Reuters、CNBC 等網站的反爬蟲驗證，達成 99% 資料擷取成功率。
 - 實作 NLP 情緒分析引擎，自動將非結構化新聞文本轉換為量化情緒分數 (Sentiment Score)。
- 高效能全端架構設計：
 - 前端採用 Next.js SSR (Server-Side Rendering) 架構，優化首屏載入速度 (FCP) 與 SEO。
 - 後端採用 FastAPI 非同步架構，處理高併發的即時報價串流與歷史回測運算。
 - 設計 RESTful API 與強型別 Schema (Pydantic)，確保前後端資料串接的穩定性與型別安全。
- 互動式資料視覺化 (Data Visualization)：
 - 使用 Recharts 開發客製化 K 線圖 (Candlestick) 與技術指標 (SMA, RSI) 圖表，支援互動式 Crosshair 與縮放功能。
 - 實作 Paper Trading (模擬交易) 系統，包含即時損益計算與持倉管理邏輯。

<https://github.com/p76121699/ai-invest-platform>[前往查看 >](#)



【專案概述】

本專案為一套高擬真的工業 4.0 模擬與監控平台，旨在解決傳統製造業缺乏即時可視化與自動化決策的問題。系統透過建立即時的數位孿生 (Digital Twin) 模型，模擬工廠生產線的運作邏輯（包含訂單生成、原料消耗、機器磨損與維護），並整合 AI 演算法實現產線的「自動駕駛」。管理者透過戰情室儀表板，不僅能即時掌握 OEE (整體設備效率)，更能依賴系統自動處理異常狀況與負載平衡。

【技術架構】

- Frontend: Next.js 14 (App Router), React, TypeScript, Tailwind CSS, Framer Motion, Recharts。
- Backend: Python, FastAPI, WebSocket, Pydantic。
- Database & AI: PostgreSQL, Google Gemini 2.0 Pro。
- DevOps & Tools: Vercel, Render, Git/GitHub。

【關鍵貢獻與技術亮點】

- 即時通訊與狀態同步 (Real-time Synchronization)：
 - 設計 WebSocket 廣播機制，成功實現後端物理引擎與前端儀表板的毫秒級同步 (Sub-100ms Latency)。
 - 實作 Data Bridge 模式，將高頻率的模擬數據流 (Telemetry) 轉換為前端可用的狀態快照 (Snapshot)，解決高併發下的效能瓶頸。
- 工廠物理仿真引擎 (Physics Simulation Engine)：
 - 開發基於 Time-Tick (時間片) 的核心模擬邏輯，精確計算機器運作中的能量消耗、零件磨損率 (Wear Rate) 以及產線阻塞 (Bottleneck) 現象。
 - 導入 動態概率算法 (Dynamic Probability Algorithm) 用於訂單生成與違約金計算，模擬真實市場的波動性與供應鏈壓力。
- AI 自主決策系統 (Autonomous System)：
 - 整合 LLM (大型語言模型) 分析工廠警報，實現系統自動診斷與修復 (Self-Healing)。例如：當偵測到輸送帶負載過高時，AI 會自動重新分配訂單或觸發預防性維護。
 - 設計 Human-in-the-loop 的權限機制，允許管理者隨時介入 (Override) 或切換回自動駕駛模式，確保系統的安全性與靈活性。
- 現代化前端互動體驗 (Modern UI/UX)：
 - 運用 Next.js SSR 優化首屏載入效能，並建立模組化的 UI Component System (卡片、警報面板、互動圖表)，提升開發維護效率。
 - 實作「戰情室」概念介面，將大量複雜數據 (KPIs, Inventory, Orders) 收斂為直觀的視覺化指標。

<https://github.com/p76121699/AI-factory-koda>

[前往查看 >](#)

【專案概述】

針對現有網路路由協定在動態流量下負載不均的問題，本研究提出了一種創新之多代理人強化學習演算法 (LS2IC, Large-scale Isomorphic Influence Communication)。透過整合 SDN 架構，實現了端對端的智慧流量工程，有效解決了傳統演算法導致的網路壅塞問題。

【核心貢獻：LS2IC 演算法開發】

- 我們提出了一種 isomorphic influence messages 機制，允許代理人對其行為對環境的影響進行學習與傳遞，從而緩解多代理系統中的異質性問題。
- 與傳統對動作進行操作的平均場方法不同，我們將平均場近似直接應用於通訊訊息，有效降低了大規模場景下的維度爆炸和計算複雜度。
- 我們引入了一種 influence reward 機制，加速有效訊息的訓練和產生。

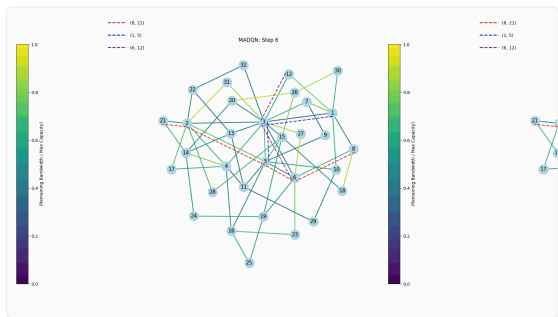
【效能評估與成果】

- Comparative Analysis: 在歐洲學術網路 (Géant) 等真實拓樸下，與傳統演算法 (OSPF) 及 SOTA DRL 模型 (MADQN, MeanField-Q) 進行壓力測試比較。
- Key Results:
 - MLU (Maximum Link Utilization) 優化：在高頻流量波動 (144 TM) 情境下，面對表現最好的基準方法，平均優勢為 7.6%，，顯顯其動態適應能力。
 - 低延遲 (Low Latency)：在高負載場景下，平均端對端路徑延遲 (Path Delay) 顯著低於其他比較算法，展現了卓越的負載平衡能力。
- 消融實驗進一步證實：只要移除影響力獎勵、輔助損失或通訊機制中的任一模組，整體性能即明顯下滑，顯示三者對最終表現皆為不可或缺的關鍵因素。

【技術堆疊】

- AI & Algorithm: PyTorch, Custom MADRL implementation (Attention, VAE, GRU)
- SDN Infrastructure: Mininet, Ryu Controller, OpenFlow 1.3
- Data Analysis: Pandas, NetworkX (Graph Theory), Seaborn (Visualization)

[前往查看 >](#)



自傳

我具備資訊工程與人工智慧相關背景，對於系統如何「實際運作並產生行為結果」特別感興趣。相較於只停留在理論或套用現成工具，我更在意模型背後的假設是否合理、訓練流程是否能真正對應問題本質，以及結果是否能被解釋與改進。

在學習與實作過程中，我長期投入於強化學習與多智能體系統的研究與實驗，曾自行設計並實作基於 Gym 介面的網格競技環境（GridWorld Battle / Predator-Prey 類型），涵蓋移動、互動、轉化與勝負條件等完整機制，並以 PyTorch 實作 Agent Policy 進行訓練。訓練過程中，我不只關注 reward 是否上升，也會主動分析行為退化、策略塌縮、探索不足等問題，並嘗試從架構（如 Actor-only、CTDE、Centralized Critic、Multi-Agent DQN）與訓練流程層面逐步修正。

這些經驗讓我培養出一個習慣：當結果不如預期時，優先檢查設計假設，而非急著調參數。例如在多智能體學習中，當出現集體偏移或無效行為時，我會回頭檢視觀測空間是否資訊不足、reward 是否誤導、critic 是否過度集中全域訊號等，並以實驗驗證推論。

除了 AI 與演算法本身，我也具備紮實的程式設計與問題拆解能力，熟悉 C++ 與 Python，能將抽象問題轉化為可驗證的邏輯結構，例如以動態規劃解決正規表示式比對問題，並進一步優化時間與空間複雜度。我對於「寫得對」與「為什麼對」同樣重視。

整體而言，我是一個偏向理性思考、重視結構與因果關係的人，學習速度建立在理解之上，而非記憶之上。未來我希望能需要深度思考、長期優化與技術累積的環境中持續成長，將模型、系統與現實問題之間的落差一步步縮小，並創造能真正產生價值的技術成果。